

ค้นสลากสิบล้านใบ และแจก แต่ละใบให้ได้เพียงครั้งเดียว

ค้นสลาก 6 หลักจำนวน 10,000,000 ใบ ด้วย pattern ที่มี wildcard แล้วแจกจ่ายให้ผู้ใช้ที่เข้ามาพร้อมกัน โดยต้องไม่มีสลากใบใดไปถึงผู้ใช้สองคน ตัวเลขทุกตัวในเอกสารนี้เป็นค่าที่วัดจริงจากระบบที่สร้างขึ้น และทดสอบภายใต้ resource limit ที่บังคับใช้จริง

01 ข้อมูลเอกสาร

ข้อมูลกำกับเอกสาร

ชื่อเอกสาร	Lottery Search System — Design Document
ผู้เขียน	Natthawat Narin, Senior Backend Developer
สถานะ	ฉบับสมบูรณ์ — ส่งเพื่อรีวิว
เวอร์ชัน	1.2
วันที่	2026-09-05
ผู้รีวิว	—
REPOSITORY	

github.com/nthw-dev/challenge-lottery-search-system — เอกสารฉบับนี้ บันทึกการตัดสินใจ ผลวิเคราะห์ และ evidence bundle ทุกชุดที่อยู่เบื้องหลังตัวเลข

PROOF OF CONCEPT

github.com/nthw-dev/poc-lottery-search-system — implementation และ loadtest ที่ให้ตัวเลขทุกตัวในเอกสารนี้

ตอบ โจทย์

`challenge-lottery-search-system.md`

บันทึกการตัดสินใจ

`adr/` — architecture decision record 8 ใบ ดูสารบัญในภาคผนวก A

ตัวเลขทุกตัวในเอกสารนี้เป็นค่าที่วัดจริงจาก proof of concept ที่ทดสอบบน Kubernetes ภายใต้ resource limit ที่บังคับใช้จริง หัวข้อ 8 เป็นสารบัญของหลักฐาน และหัวข้อ 9 ระบุสิ่งที่ยังไม่ได้พิสูจน์

02 บริบทและขอบเขต

พื้นที่ค้นหาจริงคือหนึ่งล้าน ไม่ใช่สิบล้าน

โจทย์คือ ค้นหา 6 หลักจำนวน 10,000,000 ใบ ด้วย pattern 6 ตัวอักษรที่มี wildcard * แล้วแจกจ่ายสลากที่เข้าเงื่อนไขให้ผู้ใช้ที่เข้ามาพร้อมกัน โดยต้องไม่มีสลากใบใดไปถึงผู้ใช้สองคน สิ่งที่ต้องส่งคือ solution architecture, storage สำหรับ production พร้อมเหตุผล, การวิเคราะห์ประสิทธิภาพ และกลยุทธ์ด้าน concurrency ที่ป้องกันการแจกซ้ำ หัวข้อ 4 ตอบทั้งสี่ข้อโดยตรง หัวข้อ 5 ถึง 8 ให้รายละเอียดและหลักฐาน

2.1 สมมติฐานตั้งต้น

สลาก 6 หลักมีค่าที่เป็นไปได้เพียง $10^6 = 1,000,000$ ค่า สลาก 10 ล้านใบจึงเฉลี่ยราว 10 ใบต่อหนึ่งเลข โจทย์จึงไม่ใช่ "ค้นข้อมูลสิบล้านแถว" แต่คือ "ตัดสินใจว่าเลขไหนบ้างในหนึ่งล้านเลขที่เข้าเงื่อนไข แล้วดึงสลากของเลขเหล่านั้น" พื้นที่ค้นหาแบบที่สองเล็กกว่าสิบล้าน มีขนาดคงที่ไม่ว่าจะมีสลากกี่ใบ และเล็กพอที่จะอยู่ใน cache ตลอดเวลา การตัดสินใจทุกข้อในเอกสารนี้สืบเนื่องจากจุดนี้

ขนาดของผลลัพธ์ต่างกันได้ถึงหก order of magnitudeตามจำนวน wildcard การรายงานประสิทธิภาพจึงแยกตามคลาส wildcard (หัวข้อ 6.2)

PATTERN	WILDCARD	เลขที่เข้าเงื่อนไข	สลากระบุเข้าเงื่อนไข
123456	0	1	~10
123***	3	1,000	~10,000
****23	4	10,000	~100,000
3*	5	100,000	~1,000,000
*****	6	1,000,000	10,000,000

wildcard อยู่ตำแหน่งใดก็ได้ B-tree ปกติบนคอลัมน์เลขสลากระบุจึงใช้ได้เฉพาะ pattern ที่หลักหน้าคงที่ กรณี `****23` ใช้ไม่ได้เลย ข้อจำกัดข้อนี้คือแก่นทางเทคนิคของโจทย์

03 เป้าหมายและสิ่งที่ไม่ทำ

เป้าหมายที่ทดสอบได้ 4 ข้อ ผ่านทั้งหมด และ 6 สิ่งที่ตั้งใจไม่ทำ

3.1 เป้าหมาย

ทุกเป้าหมายถูกตั้งเป็นสมมติฐานที่ทดสอบได้ก่อนสร้าง proof of concept

เป้าหมาย	เกณฑ์	ที่วัดได้	ผล
G1 คั่น wildcard ทุกตำแหน่งที่ 10M rows บน API pod 500m CPU / 128Mi	p95 < 100 ms ทุกคลาส wildcard ที่ผู้ใช้ 100 คน พร้อมกัน	p95 46.7 ms แบนราบทุกคลาส 6,068 req/s	ผ่าน (หัวข้อ 6.2)
G2 ผู้ใช้ที่คั่น pattern เดียวกันพร้อมกันไม่มีใครได้สลากระบุเดียวกัน	ซ้ำ 0 ภายใต้ contention จากผู้ใช้ 200 คน	จอง 342,412 ใบ ซ้ำ 0 ledger ไม่ตรงกับสถานะ 0	ผ่าน (หัวข้อ 8.1)

เป้าหมาย	เกณฑ์	ที่วัดได้	ผล
G3 กลยุทธ์ index ที่เลือกเร็วกว่า baseline แบบ LIKE ที่ต้นทุนพื้นที่ยอมรับได้	≥ 10 เท่าใน pattern ที่ยากที่สุด ขนาด index เทียบเคียงทางเลือกอื่น	ที่ order of magnitude ใน pattern ระบุครบ index 201 MB เทียบ 405 MB ของทางเลือกแบบรายการหลัก	ผ่าน (หัวข้อ 6.2)
G4 เสถียรภายใต้โหลดผสมต่อเนื่อง ภายใน memory limit ของ pod	ไม่ถูก OOM kill memory ต่ำกว่า 128Mi มากตลอด soak 10 นาที	942,015 request error 0 restart 0 สูงสุด 26 MiB	ผ่าน (หัวข้อ 7.3)

3.2 สิ่งที่ตั้งใจไม่ทำ

อยู่นอกขอบเขตโดยตั้งใจ เพื่อพิสูจน์ว่า database ตัวเดียวเพียงพอ ก่อนจะเพิ่มอะไรเข้าไป

- Authentication และ authorization
- Payment ระบบออกรางวัล และระบบงวด
- การขยาย database แนวนอน replication และ read replica
- Caching layer ทุกชนิด ตัด Redis ออกโดยตั้งใจ
- Observability stack ระดับ production (Prometheus, Grafana, tracing)
- CI/CD pipeline

04 สรุปการออกแบบ

โจทย์ 4 ข้อ การตัดสินใจ 4 ข้อ แต่ละข้อมีผลวัดรองรับ

คำตอบต่อสิ่งที่โจทย์ขอทั้งสี่ข้อ การตัดสินใจแต่ละข้อมีเหตุผลจากผลวัดบน proof of concept และขยายความในหัวข้อที่ระบุ

สิ่งที่โจทย์ขอ	การตัดสินใจ	เหตุผล	รายละเอียด
Solution architecture, data	API ที่ไม่เก็บ state หนึ่งตัว หน้า PostgreSQL 17 หนึ่ง instance ซึ่งมีตาราง tickets (10M แถว) ตาราง numbers แบบคงที่ (10 ⁶ แถว หนึ่งแถว	พื้นที่ค้นหาคือเลขที่เป็นไปได้ 10 ⁶ ค่า ไม่ใช่สลาก 10 ⁷ ใบ มีขนาดคงที่ ไม่ว่าจะมียุสลากรึ่ไบ และอยู่ใน cache ได้ทั้งก้อน index รวม 201	หัวข้อ 5.1 ถึง 5.3

สิ่งที่โจทย์ขอ	การตัดสินใจ	เหตุผล	รายละเอียด
structures, algorithms	ต่อค่าที่เป็นไปได้ มี index รายหลัก) และ ledger allocations แบบ append-only การค้นหาจะหาเลขที่เข้า pattern บน numbers ก่อน — หลักหน้าคงที่ → ช่วงบน primary key, ระบุครบ → point lookup, กรณีอื่น → B-tree รายหลัก — แล้ว join เข้า tickets ผ่าน partial index ของแถวที่ว่าง	MB ครึ่งหนึ่งของทางเลือกแบบรายหลัก	
Database / storage สำหรับ production	PostgreSQL 17 instance เดียว ไม่มี cache ไม่มี lock service ไม่มี search engine	FOR UPDATE SKIP LOCKED เป็นเครื่องมือแฉกงานที่ atomic และไม่มี deadlock ในตัว database เอง ข้อที่ยากที่สุดของโจทย์จึงไม่จำเป็นต้องเพิ่ม component ใด การจองและการบันทึก audit อยู่ใน transaction เดียว partial index, expression index และ planner แบบคิดต้นทุนทำให้ index พื้นที่ 10^6 ได้หลายแบบ มี component เดียวที่ต้องดูแล คาดการณ์ว่าใช้ 36 % ของ 6 core ที่เป้าตัวอย่าง 10,000 req/s	หัวข้อ 6.1, 6.4
การวิเคราะห์ประสิทธิภาพ	Strategy C: p95 46.7 ms ที่ 6,068 req/s บน pod 500m CPU ตัวเดียว แบนราบทุกคลาส wildcard เร็วกว่า LIKE สี่ order of magnitude ใน pattern ระบุครบ ทรัพยากรที่เต็มคือ API ที่ไม่เก็บ state ซึ่งขยายแนวนอนได้ trade-off ที่ยอมรับ: join เพิ่มหนึ่งชั้น (ต่ำกว่า 1 ms) ไม่มี ORDER BY ในการค้นหา และ latency ของการจองแย่งเมื่อ pattern ใกล้เคียง	วัดทั้งสีกัลยทุธบนข้อมูล 10M แถวชุดเดียวกัน C เป็นแบบเดียวที่ผ่านเกณฑ์ latency และเป็นแบบเดียวที่ย้ายคขอวดออกจาก database	หัวข้อ 6.2, 7.1, 7.2
กลยทุธ concurrency / การแจกจ่ายสลา	SQL คำสั่งเดียว: CTE เลือกแถวด้วย FOR UPDATE SKIP LOCKED อัปเดตเป็นจองแล้ว และบันทึกลง ledger เลือกแถวด้วยการสุ่มเลขจริงใน pattern และ fall back ไปสแกน pattern เมื่อไม่พบการจองหมดอายุด้วย TTL sweep	แถวที่ถูกล็อกมองไม่เห็นจาก allocator อื่นจนกว่าจะ commit ไม่มี transaction ไดรอ และเกิด deadlock ไม่ได้ พิสูจน์ด้วยการจอง 342,412 ใบภายใต้ contention จากผู้ใช้ 200 คนบน pattern เดียว	หัวข้อ 5.4, 8.1

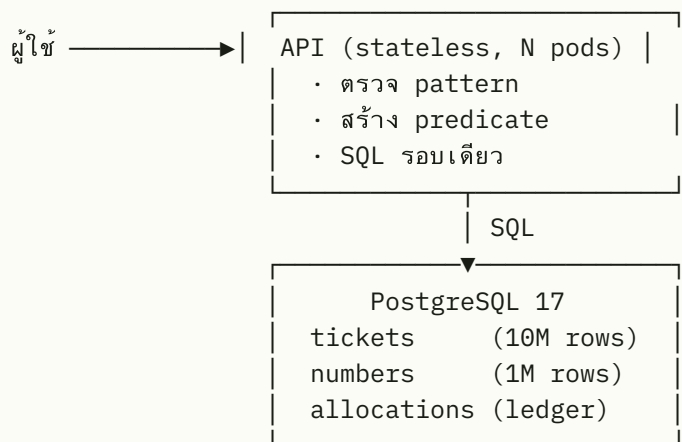
ซ้ำ 0 และ ledger ไม่ตรงกับสถานะ 0

การออกแบบคือ API ที่ไม่เก็บ state หนึ่งตัว PostgreSQL 17 หนึ่ง instance และตารางสามตาราง ไม่มีอย่างอื่น ไม่มี cache ไม่มี lock service ไม่มี search engine ตัวเลขทุกตัวในเอกสารนี้วัดบน PostgreSQL 17.11 (หัวข้อ 6.4)

05 รายละเอียดการออกแบบ

สถาปัตยกรรม โครงสร้างข้อมูล การค้นหา และ concurrency

5.1 สถาปัตยกรรมและ API



ไม่มี component อื่นอีก component ที่ไม่มีอยู่ย่อมล้มเหลวไม่ได้ ย่อมไม่มีทางข้อมูลไม่ตรงกับ database และไม่ต้องมีคณดูแล การลือกระดับแถวของ database คือตัวรับประกันเรื่อง concurrency (หัวข้อ 5.4) เหตุผลที่มักใช้ในการเพิ่ม Redis จึงไม่มี API ไม่เก็บ state จึงเพิ่ม throughput ได้ด้วยการเพิ่ม replica (หัวข้อ 7.2)

ENDPOINT

หน้าที่

POST /v1/search

ดูผลอย่างเดียว ไม่จองอะไร จำกัดผลลัพธ์ที่ 100

POST /v1/allocate

จองสลากรที่เข้าเงื่อนไขให้ผู้ใช้แบบ atomic จำกัดที่ 20

ENDPOINT	หน้าที่
POST /v1/release	คืนสลากที่จองไว้กลับเข้าระบบ
GET /healthz	liveness ตรวจสอบว่า process ยังอยู่ ไม่แตะ database (หัวข้อ 7.3)
GET /readyz	readiness ping database

pattern ต้องตรงกับ `^[0-9*]{6}$` และถูกปฏิเสธที่ชั้น handler ก่อนแตะ database กฎข้อนี้คือสิ่งที่ทำให้ตัวสร้าง predicate ใส่ตัวเลขลงใน SQL ได้อย่างปลอดภัย จำนวนผลลัพธ์เป็นค่า

ประมาณจากสูตร $10^{\text{wildcard}} \times \text{สลากต่อเลข}$ เพราะการ `COUNT(*)` จริงสำหรับ `**3**` ต้องอ่านเป็นล้านแถวเพื่อให้ได้ตัวเลขที่ไม่ต้องการความแม่นยำ

5.2 โครงสร้างข้อมูล

```
CREATE TABLE tickets (
  id          bigint    PRIMARY KEY,
  number      int       NOT NULL,          -- 0..999999
  status      smallint  NOT NULL DEFAULT 0, -- 0=ว่าง, 1=จองแล้ว, 2=ขายแล้ว
  reserved_by int,
  reserved_at timestampz
);

CREATE TABLE numbers (
  n int PRIMARY KEY,          -- พื้นที่ค้นหา 1 ล้านแถว
                                -- 0..999999
  d1 smallint NOT NULL, d2 smallint NOT NULL, d3 smallint NOT NULL,
  d4 smallint NOT NULL, d5 smallint NOT NULL, d6 smallint NOT NULL
);

CREATE TABLE allocations (
                                -- ledger แบบเขียนอย่างเดียว
  ticket_id bigint    NOT NULL,
  user_id   int       NOT NULL,
  at        timestampz NOT NULL DEFAULT now()
);
```

การตัดสินใจ	เหตุผล
number เป็น int ไม่ใช่ char(6)	ประหยัดราว 60 MB ที่ 10M rows เปรียบเทียบเร็วกว่า และดึงหลักด้วยเลขคณิตได้โดยไม่ต้องแปลง type การเติมศูนย์หน้าทำเมื่อส่งออก

ไม่เก็บหลักตัวเลขไว้บน tickets	generated column หกคอลัมน์จะกิน heap เพิ่มราว 120 MB จึงย้ายไปไว้บน ตาราง 1 ล้านแถวแทน ซึ่งต้นทุนต่ำกว่ามาก
status เป็น smallint	เล็กและเปรียบเทียบเร็วกว่า enum หรือ text เพียงพอสำหรับสามสถานะ
แยกตาราง allocations ออกมา	ทำให้การรับประกันว่าไม่แจกซ้ำตรวจสอบได้ด้วย query เดียว แทนที่จะต้อง อนุมาน (หัวข้อ 5.4, 8.1)

numbers เป็นตารางคงที่ สร้างหนึ่งล้านแถวครั้งเดียวแล้วไม่เขียนอีก และเป็นเหตุผลที่ต้นทุน
การค้นหาไม่โตตามจำนวนสลาก

5.3 อัลกอริทึมการค้นหาและ index

สำหรับ pattern อย่าง 1*3**6

1. **ตรวจ pattern** แล้วแยกตำแหน่งที่เป็นตัวเลขคงที่กับ wildcard
2. **หาเลขที่เข้าเงื่อนไข** บน numbers มีสองกรณีที่ยู่เป็นช่วงหรือจุดได้ คือหลักหน้าคงที่ติด
กัน (123***) กลายเป็นช่วงบน primary key `n BETWEEN 123000 AND 123999` และ
ระบุครบทั้งหกหลัก (123456) กลายเป็น `number = 123456` บน tickets โดยตรง
กรณีอื่นใช้ B-tree ของแต่ละหลัก
3. **ดึงสลาก** โดย join เลขเหล่านั้นเข้ากับ `tickets(number)` ผ่าน partial index ที่จำกัดไว้ที่
`status = 0`
4. **หยุดทันทีที่พอ** โดยดัน LIMIT ลงไปถึงขั้น index scan

```
-- 123*** → อ่านเป็นช่วงบน primary key ของตาราง 1 ล้านแถว
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)
LIMIT 20;

-- ****23 → ใช้ index ของแต่ละหลักบนตาราง 1 ล้านแถว
SELECT id, number FROM tickets
WHERE status = 0
      AND number IN (SELECT n FROM numbers WHERE d5 = 2 AND d6 = 3)
LIMIT 20;
```


index ทั้งหมด 7 ตัว คือ index ของหลักบน `numbers` ขนาดราว 6.8 MB ต่อตัว บวกกับ partial B-tree บน `tickets(number)` รวม **201 MB** ที่สลาก 10 ล้านใบ ส่วนที่อยู่บน `numbers` มีขนาดคงที่ มีเพียง index บนตารางสลากที่โตตามจำนวนสลาก และเพราะเป็น partial บน `status = 0` index นี้จึงเล็กลงเมื่อสลากถูกขายไป

สองกฎที่ได้จากการวัด

ไม่ใช่ `ORDER BY` ในการค้นหา `ORDER BY id LIMIT 20` เป็นต้นทุนก้อนใหญ่ที่สุดของระบบ ที่ 10M rows planner ต้องไล่อ่าน primary key แล้ว probe ตาราง `numbers` ที่ละแถว ผ่าน Memoize node ทำให้ `123***` ใช้เวลา 13.4 ms จาก 0.148 ms เมื่อไม่มีการเรียงลำดับ (90 เท่า) และคลาสแคบอื่นช้าลง 54 ถึง 92 เท่า ผลการค้นหาไม่มีลำดับที่มีความหมาย จึงตัดออก และเวลาต่อ query ลดลงต่ำกว่าหนึ่งมิลลิวินาที **ไม่ materialize ผลลัพธ์ทั้งก้อน** `**3***` เข้าเงื่อนไขราวล้านใบ แต่ผู้ใช้ต้องการยี่สิบใบ `LIMIT` จึงต้องไปถึงขั้น index scan เพื่อให้เวลาที่ได้หน้าแรกคงที่ ไม่แปรตามจำนวนที่เข้าเงื่อนไข

5.4 Concurrency และการแจกจ่ายสลาก

database เพียงตัวเดียวตอบข้อกำหนดนี้ได้

```
WITH picked AS (  
  SELECT id FROM tickets  
  WHERE status = 0 AND <pattern predicate>  
  LIMIT $2  
  FOR UPDATE SKIP LOCKED           -- ตัวรับประกัน  
) , upd AS (  
  UPDATE tickets t SET status = 1, reserved_by = $1, reserved_at = now()  
  FROM picked WHERE t.id = picked.id  
  RETURNING t.id, t.number  
) , ledger AS (  
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd  
)  
SELECT id, number FROM upd;
```

คำสั่งเดียวคือหนึ่ง transaction การเลือก การจอง และการบันทึกจึงเกิดครบทั้งหมดหรือไม่เกิดเลย `FOR UPDATE` ล็อกแถวที่เลือก ส่วน `SKIP LOCKED` ทำให้ transaction ที่เข้ามาพร้อมกัน ข้ามแถวที่ถูกล็อกไปหยิบแถวถัดไป ผลที่ตามมาสี่ข้อ คือ ผู้ใช้สองคนได้สลากใบเดียวกันไม่ได้ เพราะแถวที่ถูกล็อกมองไม่เห็นจาก allocator อื่นจนกว่าจะ commit ไม่มี lock contention

latency จึงไม่แย่งเมื่อ concurrency สูงขึ้น ไม่เกิด deadlock เพราะไม่มี transaction ใดรอ กัน และไม่ต้องมีตัวประสานงานจากภายนอก

การเลือกแถว

การหยิบตามลำดับ `id` ทำให้เกิดปัญหา hot head คือแถวที่ถูกจองแล้วกองอยู่หน้าสุดของ ลำดับ และการจองทุกครั้งต้องสแกนผ่านแถวเหล่านั้นทั้งหมด หลังจอง 713 ใบ การจองหนึ่ง ครั้งสแกนผ่าน 68,453 แถวใน 145 ms และแย่งทุกครั้ง allocator จึง **สุ่มเลขจริงใน pattern** (`****23` → เช่น `481723`) แล้วทำ point lookup ลองใหม่สูงสุดสามครั้งก่อน fall back ไป สแกน pattern ผู้ใช้ที่เข้ามาพร้อมกันกระจายไปตาม 10^{wildcard} เลข การชนกันจึงเกิดยาก และ ต้นทุนคงที่ไม่ว่าจะจองไปแล้วเท่าไร `SKIP LOCKED` เป็นตัวรับประกันความถูกต้องทั้งสองเส้น ทาง ส่วนการสุ่มเป็นมาตรการด้านประสิทธิภาพเท่านั้น

การหมดอายุของการจอง

งานเบื้องหลังปล่อยการจองที่ค้างเกินห้านาที ในระบบจริงรันเป็น scheduled job ครั้งเดียวต่อ รอบ ไม่ว่าจะ API replica ก็ตัว

หลักฐานที่วัดได้: การจอง 342,412 ใบภายใต้ contention โดยไม่มีสลากซ้ำ (หัวข้อ 8.1)
บันทึกไว้เป็น ADR-003 และ ADR-004

06 ทางเลือกที่พิจารณา

สิ่งที่วัดเทียบแล้ว และเหตุผลที่ไม่เลือก

6.1 Storage และการประสานงาน

ที่เลือก: PostgreSQL 17 instance เดียว

สิ่งที่ต้องการ	เหตุผลที่ PostgreSQL ตอบได้
แจกจ่ายสลากโดยไม่ซ้ำ	<code>FOR UPDATE SKIP LOCKED</code> เป็นเครื่องมือแจกงานที่ atomic และไม่มี deadlock ในตัว database เอง นี่คือการป้องกัน เพราะข้อที่ยากที่สุดของโจทย์เป็นพีเจอร์บรรทัดเดียวของ database
ประสิทธิภาพการค้น wildcard	expression index, partial index และ planner แบบคิดต้นทุน ทำให้ index พื้นที่ค้นหาหนึ่งล้านเลขได้หลายรูปแบบ วัดได้ p95 47 ms ที่ 10M rows ภายใต้ผู้ใช้ 100 คนพร้อมกัน

สิ่งที่ต้องการ	เหตุผลที่ PostgreSQL ตอบได้
ความถูกต้องเชิง transaction	การจองสลากรับการเขียน ledger อยู่ใน transaction เดียวกัน ซึ่ง lock service ที่แยกออกไปให้ไม่ได้
ดูแลง่าย	มี component เดียวที่ต้อง deploy, backup, monitor และทำความเข้าใจ พื้นที่รวมสำหรับสลาก 10 ล้านใบพร้อม index ทั้งหมดราว 1.4 GB
ยังโตต่อได้	โมเดลต้นทุนที่วัดได้ (หัวข้อ 7.2) คาดว่า instance เดียวใช้ 36 % ของ 6 core ที่เป้าตัวอย่าง 10,000 req/s

ทางเลือกที่ไม่เลือก

ทางเลือก	เหตุผลที่ไม่เลือก
ใส่ Redis เป็น cache ข้างหน้า	เพิ่ม component และ failure mode และสร้างปัญหาความไม่สอดคล้องตรงจุดที่ความถูกต้องสำคัญที่สุด การจองต้องเขียนลง database อยู่ดี cache จึงเป็นเจ้าของการตัดสินใจไม่ได้
Redis distributed lock	ต้องจัดการ lock expiry รับมือกับ process ที่หยุดทำงานขณะถือ lock และยังคงเขียน database อยู่ดี SKIP LOCKED ให้การรับประกันเดียวกันในระดับ transaction โดยไม่มีต้นทุนเพิ่ม
SELECT FOR UPDATE ที่ไม่มี SKIP LOCKED	ถูกต้องแต่ทำให้ทุก allocator ต้องเรียงคิวรอตัวแรก latency เป็นวินาทีทันทีที่เกิด contention
Optimistic locking ด้วย version column	เมื่อผู้ใช้จำนวนมากแข่งกันจอง pattern ยอดนิยม อัตราการ retry จะสูงขึ้นจน throughput ลดลงอย่างรุนแรง
ล็อกในหน่วยความจำของ application	ใช้ข้าม replica ไม่ได้ ขณะที่การเพิ่ม API replica คือเส้นทางขยายหลัก (หัวข้อ 7.2)
Elasticsearch หรือ search engine	แก้ปัญหาการค้นข้อความ ไม่ใช่ key space ตายตัวหลัก และไม่ช่วยเรื่องการจองแบบ atomic ซึ่งเป็นความยากที่แท้จริง
คำนวณผลของทุก pattern ไว้ล่วงหน้า	pattern ที่เป็นไปได้ 10^6 แบบ และผลลัพธ์บางอันมีเป็นล้านแถว ต้นทุนพื้นที่เก็บ และการล้าง cache สูงกว่าต้นทุน query ที่เลี่ยงได้

บันทึกไว้เป็น ADR-001 และ ADR-003

6.2 กลยุทธ์ index

สร้างและวัดทั้งสี่กลยุทธ์บนข้อมูล 10 ล้านแถวชุดเดียวกัน กลยุทธ์เดียวโดยไม่มีตัวเปรียบเทียบ
พิสูจน์ได้แค่ว่าทำงานได้ ไม่ได้พิสูจน์ว่าเป็นตัวเลือกที่ถูก

กลยุทธ์	ทำอย่างไร	ขนาด INDEX
O Baseline	<code>lpad(number::text,6,'0') LIKE '____23'</code>	ไม่มีที่ใช้ได้
A Expression index	สร้าง partial index แยกที่ละหลักบน <code>tickets</code> แล้วให้ planner รวมผลเอง	405 MB
B Dimension table	หาเลขที่เข้าเงื่อนไขจาก <code>numbers</code> ก่อน แล้ว join	201 MB
C B + prefix range	เหมือน B แต่หลักหน้าที่คงที่กลายเป็นช่วงบน key และ pattern ที่ระบุครบ กลายเป็น point lookup	201 MB

ต้นทุนต่อ query เดียว หน่วยเป็นมิลลิวินาที (PostgreSQL 17.11, cache อุ่นแล้ว)

PATTERN	WILDCARD	O	A	B	C
<code>123456</code>	0	829	130	5.1	0.087
<code>123***</code>	3	2.5	21.6	1.7	0.148
<code>12*4**</code>	3 กระจาย	1.5	21.6	1.3	0.158
<code>****23</code>	4	0.17	0.11	0.22	0.132
<code>**3***</code>	5	0.05	0.05	0.33	0.206
<code>*****</code>	6	0.04	0.02	0.02	0.024

baseline ดูใกล้เคียงในแถวล่างเพียงเพราะ `LIMIT 20` ทำให้การสแกนหยุดได้เร็วเมื่อสลากที่เข้าเงื่อนไขมีมาก แถวที่ตัดสินคือแถวระบุครบ ซึ่งมีสลากเข้าเงื่อนไขราวสิบใบจากสิบล้าน ตรงนั้นแบบที่เลือกเร็วกว่าคือ **order of magnitude** (ราว 9,500 เท่าในรอบนี้ ค่าจากการรันครั้งเดียวผันแปรได้ในแต่ละรอบ ระดับ order of magnitude คือข้อความที่ยืนยันได้)

ภายใต้โหลด ผู้ใช้ 10 ถึง 100 คน บน API pod ขนาด 500m CPU ตัวเดียว

กลยุทธ์	P95 W0	P95 W3	P95 W4	P95 W5	P95 รวม	REQ/S	ล้มเหลว
C	46.8	46.9	46.5	46.7	46.7	6,068	0 %

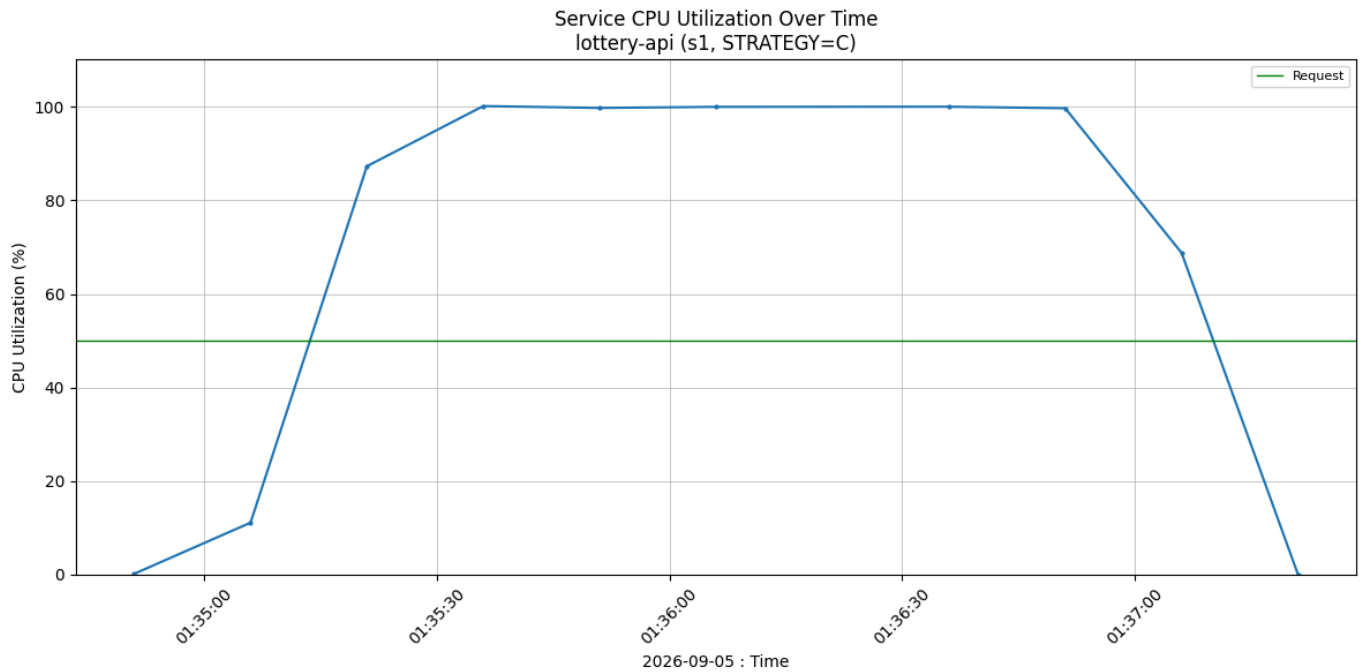
กลยุทธ์	P95 W0	P95 W3	P95 W4	P95 W5	P95 รวม	REQ/S	ล้มเหลว
B	176.0	114.1	103.3	103.6	108.3	975	0 %
A	5,663	4,503	4,503	4,392	4,705	25	0 %
O	1,320	1,049	1,018	1,053	1,051	128	94.3 %
ภายใต้ Strategy C ค่า latency แบนราบเท่ากันทุกคลาส wildcard ซึ่งเป็นคุณสมบัติที่หัวข้อ 2.1 ต้องการ							

แต่ละกลยุทธ์ใช้ทรัพยากรตรงไหน

กลยุทธ์	CPU ของ API	CPU ของ POSTGRES	MEMORY ที่ POSTGRES เหลือ
C	100 % ถึง limit	85 %	48 MiB
B	24 %	102 % ถึง limit	17 MiB
A	2 %	102 % ถึง limit	25 MiB
O	1 %	101 % ถึง limit	235 MiB

ตารางนี้คือตัวตัดสินใจการออกแบบ **Strategy C** เป็นแบบเดียวที่ย้ายคอขวดออกจาก **database** Postgres ยังมีที่ว่างเหลือ throughput จึงเพิ่มได้ด้วยการเพิ่ม API replica ที่ไม่เก็บ state ส่วน A กับ B นั้น database คือข้อจำกัด การเพิ่ม API ไม่ช่วย

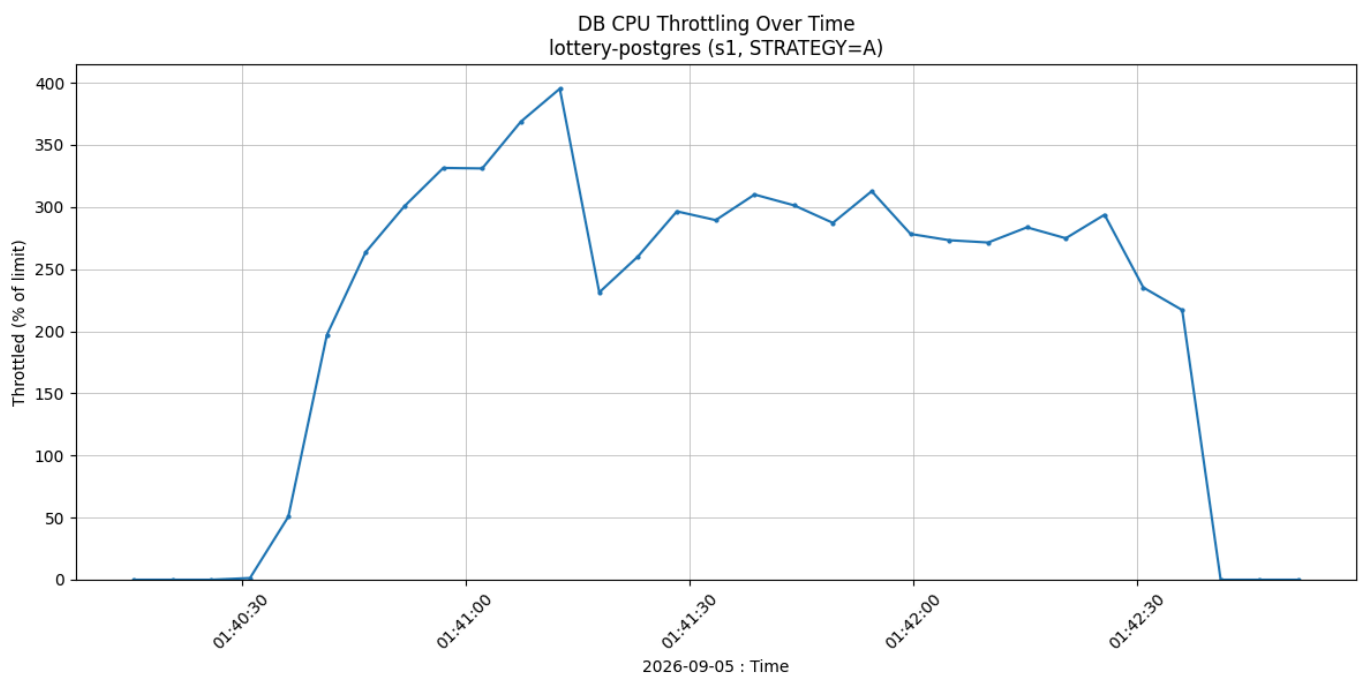
memory ของ Postgres ริงขีด limit 1 GiB ในทุกกลยุทธ์ที่มี index (working set 976 ถึง 1,007 MiB สำหรับ A, B และ C) เพราะ hot set ของตาราง 10 ล้านแถวพร้อม index มีขนาดพอกับ pod ไม่มี pod ใดถูก OOM kill แต่ profile แบบ baseline มีระยะเผื่อน้อย ซึ่งเป็นเหตุผลหนึ่งที่หัวข้อ 7.2 จัดสรร 3Gi



ภายใต้ Strategy C ตัวที่เติมก่อนคือ API pod ทรัพยากรที่เป็นข้อจำกัดคือตัวที่ทำสำเนาเพิ่มได้ เส้นสีเขียวคือค่า CPU request ของ pod

ทำไม Strategy A ถึงล้มเหลว

Strategy A ทำ index ทุกหลักแล้วให้ planner ตัดกันเอง ตัวเลขหลักเดียวคัดแหวทิ้งได้เพียง 10 % index บนหลักเดียวจึงยังชี้ไปที่หนึ่งล้านแหวจากสิบล้านแหว การรวม index หกตัวมีต้นทุนสูงกว่าที่ประหยัดได้ planner จึงใช้ index ตัวเดียวแล้ว filter ที่เหลือ



Strategy A ทำให้ database ถูกตัดโควตา CPU สูงสุดถึง 395 % ต่อวินาที และ index ขนาด 405 MB ให้ throughput เพียง 25 req/s ขนาดของ index กับความสามารถในการคัดแถวทั้งเป็นคนละคุณสมบัติ index ที่คัดแถวทั้งได้ไม่มากมีต้นทุนสูงกว่าการสแกนทั้งตาราง

บันทึกไว้เป็น ADR-002

6.3 Strategy D – วัดแล้วและตัดสินใจไม่ใช่

PostgreSQL 17 ค้น `= ANY(array)` ได้ด้วย B-tree descent เพียงครั้งเดียว ซึ่งเปิดทางให้เขียน query อีกแบบ คือให้ API คำนวณเลขที่เข้าเงื่อนไขเองแล้วส่งเป็น `int[]` แทน subquery บน `numbers` Strategy D ใช้กับ pattern ที่มี wildcard ไม่เกินสามตัว (1,000 คำราร 4 KB) pattern ที่กว้างกว่านั้น fall back ไปใช้ C

วัดด้วยคลาส `w0` (ระบุครบ) `w3lead` (`ddd***`) และ `w3scatter` (`dd*d**`) คลาสสุดท้ายไม่มีหลักหน้าติดกัน C จึงไม่มีช่วง key ให้ใช้ และเป็นคลาสเดียวที่ D มีโอกาสต่างจาก C อย่างมีนัยสำคัญ (`d1-C/`, `d1-D/`)

STRATEGY	P95 รวม	สูงสุด	THROUGHPUT	CPU ของ API	CPU ของ DB
C	48.4 ms	153 ms	4,856 req/s	100 %	71 %
D	55.7 ms	77 ms	6,110 req/s	100 %	35 %

ทั้งสองรอบ HTTP error 0 สลากเข้า 0 และ ledger ไม่ตรงกับสถานะ 0 ค่า p95 แยกคลาสแบนราบต่างกันไม่เกิน 0.3 ms ทั้งสองแบบ (C: 48.4 / 48.4 / 48.6 ms, D: 55.5 / 55.7 / 55.7 ms)

D ย้ายงานจาก database ไปที่ API คือ CPU สูงสุดของ database ลดจาก 71 % เหลือ 35 % throughput เพิ่ม 26 % p95 แล่ง 15 % และ API เป็นทรัพยากรที่เต็มในทั้งสองรอบ query plan เดียวใน `docs/explain-k8s-10M.txt` ให้ภาพเดียวกัน คือ D เร็วกว่าเล็กน้อยใน pattern ที่มี wildcard สามตัว และแยกไม่ออกในกรณีอื่น มีข้อสรุปสองข้อ **สิ่งที่ D ให้คือ headroom ของ database ไม่ใช่ความเร็วที่ผู้ใช้รู้สึกได้** เพราะสิ่งที่ประหยัดได้มาจากการตัด join กับ `numbers` ขณะที่สิ่งที่ผู้ใช้รอคือ API pod ที่เต็มอยู่ และ **throughput กับ latency วิ่งสวนทางกัน** คือ request ผ่าน pod เดิมได้มากขึ้น แต่ tail latency ไม่ดีขึ้น

ไม่ใช่ Strategy D หัวข้อ 7.2 ระบุว่า database ใช้ 36 % ของที่จัดสรรไว้ที่เป้าตัวอย่าง 10,000 req/s headroom ของ database จึงแก้ปัญหาที่ระบบนี้ไม่มี ขณะที่ D เพิ่มเส้นทาง

predicate อีกชุด ต้อง allocate หน่วยความจำสำหรับเลขสูงสุด 1,000 ตัวต่อ request และเพิ่ม traffic อีก 4 KB ต่อ query แคบ D จะเป็นคำตอบที่ถูกก็ต่อเมื่อ database กลายเป็นข้อจำกัด ซึ่งผลวันนี้ชี้ว่าในกรณีนั้น D จะลด CPU ของ database ลงครึ่งหนึ่ง โค้ดยังอยู่ใน proof of concept ภายใต้อ STRATEGY=D บันทึกไว้เป็น ADR-008

6.4 เวอร์ชันของ PostgreSQL: 17 ทั้งหมด

ตัวเลขทุกตัวในเอกสารนี้วัดบน **PostgreSQL 17.11** (postgres:17-alpine) บนข้อมูล 10 ล้านแถวชุดเดียว profile แบบ baseline ชุดเดียว (API pod เดียวขนาด 500m คู่กับ database 2000m/1Gi) และสคริปต์เก็บหลักฐานชุดเดียว ทุก evidence bundle บันทึก pg_version=17.11 ไว้ใน 41-run.txt ไม่มีตัวเลขใดถูกยกมาหรือปรับสัดส่วนจากเวอร์ชันอื่น

ใช้เวอร์ชัน 17 เพราะเป็นเวอร์ชันที่วัดจริง และการออกแบบไม่พึ่งพาสิ่งใดนอกเหนือจากนั้น การรับประกันว่าไม่แจกซ้ำอาศัย SKIP LOCKED เส้นทางค้นหาอาศัย partial B-tree index และช่วงบน primary key ซึ่งทั้งหมดทำงานเหมือนกันบน 16 ความสามารถเฉพาะของ 17 ข้อเดียวที่พิจารณา คือการค้น = ANY(array) ด้วย B-tree descent เพียงครั้งเดียว เป็นเหตุให้วัด Strategy D ซึ่งหัวข้อ 6.3 ตัดสินใจไม่ใช่ การอัปเดต major version ครั้งถัดไปเป็นการขยับเวอร์ชันโดยไม่ต้องแก้ schema หรือ SQL แต่ควรรันชุด loadtest ซ้ำ แทนที่จะถือว่าตัวเลขเดิมนั้นยังใช้ได้ เพราะ benchmark แบบ query เดียวกับแบบมีคนใช้พร้อมกันตอบคนละคำถาม บันทึกไว้เป็น ADR-007

07 ประเด็นร่วมของทั้งระบบ

trade-off ด้านประสิทธิภาพ การขยายระบบ การดูแลระบบ และ observability

7.1 trade-off ด้านประสิทธิภาพที่ยอมรับ

- join เพิ่มหนึ่งชั้น Strategy C อ่านสองตาราง ต้นทุนที่วัดได้คือเศษเสี้ยวของมิลลิวินาที เพราะตาราง numbers อยู่ใน cache
- ตารางที่ต้อง seed เพิ่มหนึ่งตาราง คำสั่งเดียว รันครั้งเดียว

- **wildcard ที่อยู่หน้าไม่ได้ประโยชน์จาก prefix range** `****23` fall back ไปใช้ index รายหลักที่ 0.13 ms จึงไม่ใช่เส้นทางที่อ่อนแอ
- **การจองช้าลงเมื่อสลากริโกล์หมด** เมื่อ `****23` ถูกจองไป 96 % ค่า p95 ขึ้นจาก 75 ms เป็น 510 ms เพราะการสุ่มเลขพบสลากริโกล์ที่ถูกจองแล้วบ่อยขึ้น การถอน pattern ที่สลากริโกล์หมดเป็นการตัดสินใจระดับ product ไม่ใช่ปัญหาของ database แต่ต้องออกแบบเพื่อไว้

7.2 การวางแผนขยาย

เป้าตัวอย่าง

เป้า 10,000 request ต่อวินาที และเพดานการใช้ทรัพยากร 60 % เป็นค่าที่ผู้เขียนกำหนดขึ้น เพื่อสาธิตวิธีคิด capacity โจทย์ไม่ได้ระบุทั้งสองค่า โมเดลด้านล่างใช้กับเป้าอื่นได้โดยไม่ต้องเปลี่ยนอะไร

เมื่อต้นทุนต่อ request คงที่ capacity จึงวางแผนด้วยการคำนวณได้ การทดสอบแบบขั้นบันได ส่งคำขอที่ 1,000 / 2,000 / 3,000 / 4,000 request ต่อวินาที ขึ้นละ 60 วินาที บน profile ที่เสนอคือ API 4 pods ขนาด 500m กับ database 6 core วัดเฉพาะช่วง 50 วินาทีที่อัตรานิ่งของแต่ละขั้น

อัตราคำขอ	CPU ของ API (4 PODS)	API % ของ LIMIT	CPU ของ POSTGRES	POSTGRES % ของ LIMIT
1,000	100m	5.0 %	320m	5.3 %
2,000	192m	9.6 %	484m	8.1 %
3,000	293m	14.6 %	706m	11.8 %
4,000	415m	20.7 %	942m	15.7 %

k6 ส่งคำขอได้ 579,999 จาก 580,000 ครั้งที่ส่ง (100.00 %) ไม่มี request ล้มเหลว ไม่มี dropped iteration และ p95 1.12 r เท่ากันคือสิ่งที่ทำให้ใช้โมเดลเส้นตรงได้

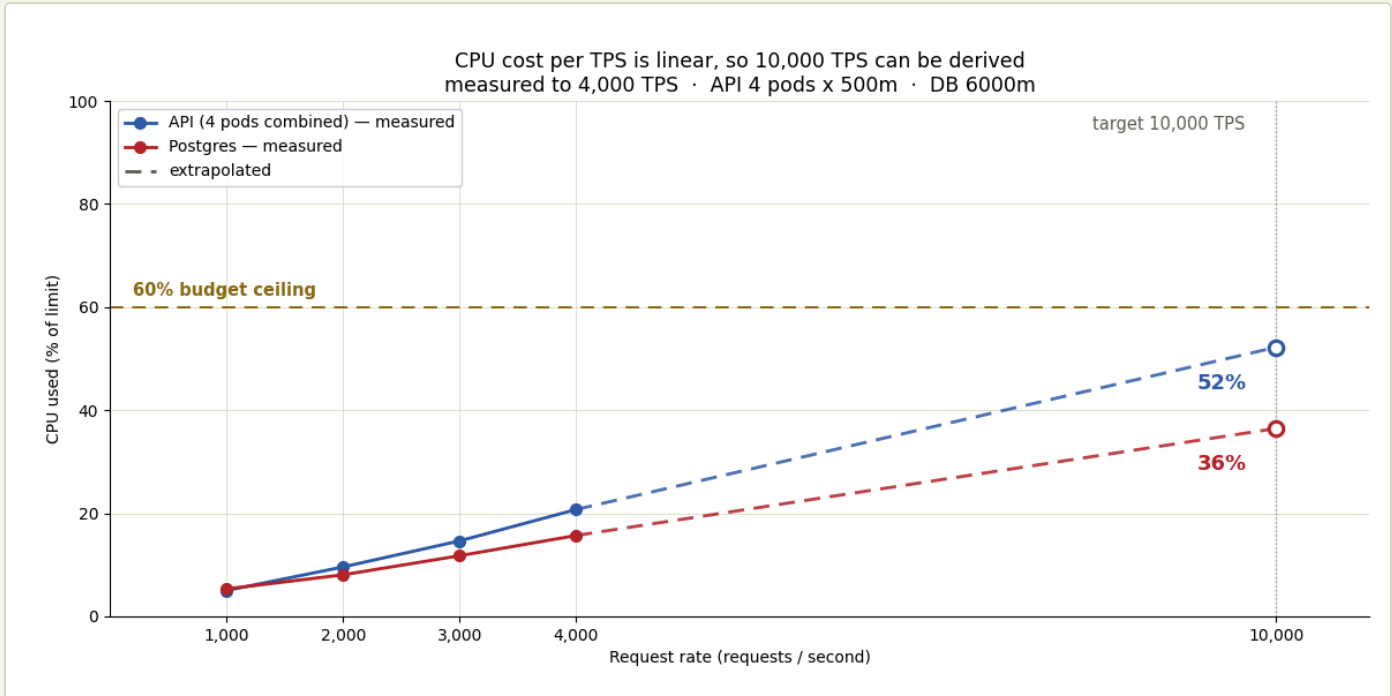
CPU ที่ใช้ = งานพื้นหลัง + ต้นทุนต่อ request × อัตราคำขอ

API = -4.6 m-core + 0.1049 m-core ต่อ req/s → 1,044m ที่ 10,000 req/s

Postgres = 112.1 m-core + 0.2075 m-core ต่อ req/s → 2,187m ที่ 10,000 req/s

การลากเส้นผ่านขั้นแรกกับขั้นสุดท้ายแยกงานพื้นหลังที่คงที่ออกจากต้นทุนต่อ request ค่าตัดแกนของ API ที่ติดลบเล็กน้อย เป็นผลข้างเคียงของการลากเส้นผ่านสองจุด API แทบไม่มีการพื้นหลัง ต้นทุนเกือบทั้งหมดแปรตามโหลด ที่เป้า 10,000 request ต่อวินาที โดยให้ทั้งสองฝั่งใช้ทรัพยากรไม่เกิน 60 % ของ limit

ส่วนประกอบ	REPLICAS	CPU LIMIT	MEMORY LIMIT	คาดว่าจะใช้ที่ 10,000 REQ/S
API ไม่มี state จึงขยาย แนวนอน	4	500m ต่อ pod	128Mi ต่อ pod	CPU 52 % · memory 13 %
PostgreSQL ตัวเดียว	1	6,000m	3Gi	CPU 36 % · memory 44 %



เส้นทึบคือค่าที่วัดจริง เส้นประคือค่าที่คำนวณต่อ เส้นแนวนอนคือ budget 60 % ที่ 10,000 request ต่อวินาที ทั้งสองฝั่งยังอยู่ใต้เส้น การพล็อตเป็นเปอร์เซ็นต์ของ limit แทนค่า CPU ดิบ ทำให้ส่วนประกอบสองตัวที่ขนาดต่างกันใช้เส้น budget เดียวกันได้

มีสามเรื่องที่ไม่เคลซึ่งมองแต่ CPU จะมองข้าม **memory** ของ **database** ไม่ได้โตเป็นเส้นตรงตามโหลด แต่โตตามข้อมูลที่ถูกดึงเข้า cache แล้วแบนราบ ส่วนต่างของแต่ละชั้นคือ 232 → 174 → 41 MiB ลู่อเข้าหาราว 1,363 MiB ถ้าลากเส้นตรงจะได้ราว 2,217 MiB และที่ limit 2,560Mi ที่ใช้ทดสอบจริง จุดอิ่มตัวอยู่ที่ 53 % ยังอยู่ใน budget แต่ไม่มีระยะเผื่อ จึงแนะนำ 3Gi (44 %) **connection pool** ไม่ใช่ข้อจำกัด จำนวนที่ต้องใช้พร้อมกันคือ อัตราคำขอ ×

เวลาต่อคำขอ = $10,000 \times 0.69 \text{ ms} \approx 7 \text{ connection}$ จากที่เตรียมไว้ 40 ขยาย **API แนว**
นอน ไม่ใช่แนวตั้ง เพราะ Go runtime ถูกตรึงไว้ที่หนึ่ง processor ต่อ pod การทำ pod ให้
ใหญ่ขึ้นไม่ช่วย แต่การเพิ่มจำนวน pod ช่วย และได้ความทนทานเพิ่ม

เมื่อเกินราว 20,000 request ต่อวินาที database ตัวเดียวจะกลายเป็นข้อจำกัด ขั้นถัดไปคือ
read replica สำหรับการค้นหาอย่างเดียว การจองยังส่งเข้า primary เพราะ **SKIP LOCKED**
ต้องการข้อมูลชุดที่เป็นต้นฉบับ

7.3 ความทนทานและการดูแลระบบ

liveness ต้องไม่ขึ้นกับ **database** เมื่อใช้ baseline strategy connection pool เต็มไปด้วย
query เข้า readiness probe ตอบไม่ทัน Kubernetes ถอด pod ออกจาก Service และ
15,592 จาก 16,528 request ล้มเหลว ซึ่งเป็นพฤติกรรมที่ถูกต้อง แต่ถ้า *liveness* probe
แตะ database ด้วย Kubernetes จะ restart pod ที่ยังติดอยู่ระหว่างที่ database เข้า และ
เปลี่ยน degradation ให้กลายเป็น outage **/healthz** จึงตรวจแค่ process ส่วน **/readyz**
ping database บันทึกไว้เป็น ADR-006

resource limit ต้องไปถึง **runtime** process ของ Go อ่านจำนวน core ของเครื่อง ไม่ใช่
โควตาของ cgroup จึงตั้ง **GOMAXPROCS** และ **GOMEMLIMIT** ให้ชัดเจน ถ้าไม่ตั้ง runtime จะ
สร้าง thread มากเกินจริงและถูก OOM kill ในที่สุด

ความเสถียร การรันต่อเนื่อง 10 นาทีแบบผสมทั้งค้นหา จอง และคืน ได้ 942,015 request
ไม่มี error ไม่มี restart ไม่มี OOM kill และ memory ของ API สูงสุดเพียง 26 MiB จาก limit
128 MiB

7.4 Observability

สัญญาณที่ต้องมีใน production: query plan ผ่าน **pg_stat_statements** lock
contention ผ่าน **pg_locks** และ **pg_stat_activity** CPU และ memory ของแต่ละ pod
เทียบกับ limit และ CPU throttling ซึ่งกราฟ CPU เฉลี่ยมองไม่เห็น และเป็นกลไกที่ทำให้
Strategy A ล้มเหลว (หัวข้อ 6.2) observability stack เต็มรูปแบบเป็นสิ่งที่ตั้งใจไม่ทำ (หัวข้อ
3.2)

ทุกข้อความ และตรวจสอบได้จากที่ไหน

8.1 หลักฐานจาก loadtest ว่าไม่แจกซ้ำ

ทดสอบสองรอบ แต่ละรอบมีผู้ใช้ 200 คนส่งคำขอ pattern เดียวกันพร้อมกันเป็นเวลา 60 วินาที

	****23 สลากราว 100K ใบ	**3*** สลากราว 1M ใบ
จองได้ใน 60 วินาที	95,630	246,782
แถวใน ledger = สลากที่ถูกจอง = id ที่ไม่ซ้ำ	95,630	246,782
สลากที่ถูกแจกสองครั้ง	0	0
ledger ที่ไม่ตรงกับสถานะสลาก	0	0
HTTP error	0	0
จองรวม 342,412 ใบ ซ้ำ 0 ใบ ตรวจสอบด้วย query เดียวที่ต้องคืนศูนย์แถว <code>SELECT ticket_id, count(*) FROM allocations GROUP BY ticket_id HAVING count(*) > 1;</code>		

8.2 Automated test

เงื่อนไขเดียวกันถูกยืนยันใน `go test` กับ PostgreSQL จริง คือ 50 goroutine แข่งกันจอง สลาก 120 ใบ ทุกใบถูกแจกครั้งเดียวพอดี และเมื่อของหมดได้ผลลัพธ์ว่าง ไม่ใช่ error ชัด ทดสอบรันกับทุกกลยุทธ์ index รวมทั้งที่ถูกตัดทิ้ง ความถูกต้องจึงไม่ขึ้นกับเส้นทางการค้นหา

8.3 สารบัญหลักฐาน

ข้อความ	แหล่งอ้างอิง
query plan และขนาด index	<code>docs/explain-k8s-10M.txt</code>
ต้นทุนของแบบที่ใช้ <code>ORDER BY</code> ซึ่งถูกตัดทิ้ง	<code>docs/explain-k8s-10M-orderby.txt</code>
การเปรียบเทียบกลยุทธ์ภายใต้โหลด	<code>docs/evidence/s1-C/</code> , <code>s1-B/</code> , <code>s1-A/</code> , <code>s1-0/</code>

ข้อความ	แหล่งอ้างอิง
การแจกจ่ายสลากโดยไม่ซ้ำ	<code>docs/evidence/s2/30-verify.txt</code> , <code>s2b/30-verify.txt</code>
ความเสถียรตลอด 10 นาที	<code>docs/evidence/s3/</code>
โมเดล capacity สำหรับเป้าตัวอย่าง 10,000 req/s	<code>docs/evidence/c1-capacity/50-capacity.json</code>
Strategy D เทียบกับ Strategy C	<code>docs/evidence/d1-C/</code> , <code>d1-D/</code>
เวอร์ชัน PostgreSQL ของทุกรอบ	<code>pg_version=17.11</code> ใน <code>41-run.txt</code> ของทุก bundle
บทวิเคราะห์ทุกรอบการทดสอบ	<code>docs/RESULTS.md</code>

แต่ละ bundle ประกอบด้วย log ของload generatorแบบเต็ม summary ทั้งแบบ JSON และ HTML กราฟทรัพยากร ข้อมูลดิบที่ใช้วาดกราฟ และผลตรวจ SQL

09 ความเสี่ยงและข้อจำกัด

สิ่งที่การออกแบบนี้ยังไม่ได้พิสูจน์

- **10,000 req/s** มาจากการคำนวณ ไม่ใช้การวัด อัตราสูงสุดที่วัดจริงคือ 4,000 req/s เป้าตัวอย่างจึงเป็นการลากเส้นตรงต่อไปอีก 2.5 เท่า ข้อมูลที่จุดสนับสนุนว่าเป็นเส้นตรง แต่ไม่ได้พิสูจน์ว่าจะจริงต่อไป ก่อนขึ้น production ควรทดสอบจริงด้วย load generator บนเครื่องที่แยกจาก cluster
- พฤติกรรมเมื่อสลากใกล้หมดยังเข้าใจเพียงบางส่วน latency ของการจองแย่งเมื่อ pattern ใกล้ขายหมด แต่ยังไม่ได้กำหนดว่าควรถอน pattern ออกจากการขายที่จุดไหน
- การวัดทั้งหมดมาจาก **node เดียว** ซึ่ง API, database และload generatorใช้ CPU ชุ่เดียวกัน throughput สัมบูรณ์บนเครื่องที่แยกกันจะต่างออกไป แต่การเปรียบเทียบระหว่างกลยุทธ์ ซึ่งเป็นฐานของข้อเสนอจะ ไม่เปลี่ยน
- **ยังไม่ได้ทดสอบ failure mode** ไม่มีการวัดใดครอบคลุมกรณี pod หาย rolling update หรือ failover ของ database ซึ่งเป็นช่วงที่ capacity ลดลง จึงควรเผื่อ replica เพิ่มอีกอย่างน้อยหนึ่งตัวจากที่คำนวณได้

- ยังไม่ได้ทดสอบเมื่อสลาถูกจองไปแล้วเป็นส่วนใหญ่ การวัดทั้งหมดเริ่มจากสลาที่ว่างทั้งหมด ถ้าสลาถูกขายไปแล้ว 80 % partial index จะเล็กลงแต่โอกาสที่การสุ่มเลขจะพบสลาว่างก็ลดลง ทั้งสองผลต้องวัดจริง
- สิ่งที่ตั้งใจไม่รวมไว้ authentication, payment, ระบบงวด, replication และ caching layer (หัวข้อ 3.2)

10 ภาคผนวก

บันทึกการตัดสินใจ คำศัพท์ และการเชื่อมกับเกณฑ์ให้คะแนน

A. Architecture decision records

หนึ่งบันทึกต่อการหนึ่งการตัดสินใจ อยู่ใน adr/ แต่ละใบระบุบริบท การตัดสินใจ ทางเลือกที่ปฏิเสธ ผลที่ยอมรับ และหลักฐานที่สนับสนุน

ADR	การตัดสินใจ	หัวข้อ
ADR-001	ใช้ PostgreSQL ตัวเดียว ไม่มี cache ไม่มี lock service	6.1
ADR-002	หาเลขที่เข้า pattern บนตาราง numbers 1 ล้านแถวก่อน แล้ว join (Strategy C)	5.3, 6.2
ADR-003	กันแจกซ้ำด้วย FOR UPDATE SKIP LOCKED ในคำสั่งเดียว	5.4
ADR-004	จองด้วยการสุ่มเลขจริงก่อน แล้วจึง fall back ไปสแกน pattern	5.4
ADR-005	ไม่ใส่ ORDER BY ในการค้นหา	5.3
ADR-006	liveness ต้องไม่แตะ database ส่วน readiness ต้องแตะ	7.3
ADR-007	ใช้ PostgreSQL 17	6.4
ADR-008	ไม่ใช่ Strategy D	6.3

B. คำศัพท์

คำศัพท์	ความหมายในเอกสารนี้
p95	เวลาที่ 95 % ของ request เร็วกว่าค่านี้ ใช้แทนค่าเฉลี่ยเพราะค่าเฉลี่ยซ่อนหางซ้ายที่ผู้ใช้รู้สึกได้จริง
VU	virtual user คือผู้ใช้จำลองหนึ่งคนใน load generator ที่ส่ง request วนซ้ำ 200 VUs คือจำลอง 200 คนพร้อมกัน
คลาส wildcard	การจัดกลุ่ม pattern ตามจำนวน * (w0, w3, w4, w5) ต้องรายงาน latency แยกตามคลาส เพราะขนาดผลลัพธ์ต่างกันถึงหก order of magnitude
SKIP LOCKED	ตัวเลือกของ <code>SELECT ... FOR UPDATE</code> ที่สั่งให้ transaction ข้ามแถวที่ transaction อื่นถืออยู่แทนที่จะรอ เป็นฐานของการรับประกันว่าไม่แจกซ้ำ
BitmapAnd	วิธีที่ planner รวมผลจากหลาย index จะคุ้มก็ต่อเมื่อแต่ละ index คัดแถวทิ้งได้มาก index ของเลขหลักเดียวคัดได้แค่ 10 % จึงเป็นเหตุที่ Strategy A ล้มเหลว
partial index	index ที่จำกัดด้วยเงื่อนไข <code>WHERE</code> ในที่นี้คือ <code>WHERE status = 0</code> จึงครอบคลุมเฉพาะสลากรที่ว่าง และเลิกลงเมื่อสลากรถูกขายไป
working set	memory ที่ใช้จริงโดยไม่นับ page cache ที่ระบบเรียกคืนได้ ตัวเลข memory ดิบของ container PostgreSQL จะอยู่ใกล้ 100 % ของ limit ตลอด เพราะ PostgreSQL ใช้พื้นที่ว่างเป็น page cache working set คือตัวเลขที่มีความหมาย
CPU throttling	เวลาที่ container ถูกปฏิเสธ CPU เพราะใช้เกินโควตา มองไม่เห็นในกราฟ CPU เฉลี่ย ต้องวัดตรงจาก cgroup
liveness / readiness	liveness ถามว่า process ยังมีชีวิตไหม ถ้าไม่ผ่านจะถูก restart ส่วน readiness ถามว่ารับงานได้ไหม ถ้าไม่ผ่านจะถูกถอดจาก load balancer โดยไม่ restart

C. การเชื่อมกับเกณฑ์ให้คะแนนของโจทย์

เกณฑ์	ตอบไว้ที่
Feasibility	หัวข้อ 5.1 สถาปัตยกรรม, หัวข้อ 5.2 โครงสร้างข้อมูล, หัวข้อ 5.3 อัลกอริทึม ซึ่งสร้างขึ้นจริงและวัดผลแล้ว
Performance	หัวข้อ 6.2 การเทียบสืแบบที่ 10M rows และหัวข้อ 7.2 โมเดลการขยาย
Correctness	หัวข้อ 5.4 การออกแบบเรื่อง concurrency และผลตรวจการจอง 342,412 ใบในหัวข้อ 8.1

เกณฑ์	ตอบไว้ที่
Real-world practicality	หัวข้อ 6.1 การเลือก database และทางเลือกที่ไม่เลือก, หัวข้อ 7.3 การดูแลระบบ, หัวข้อ 9 ข้อจำกัด
Creativity	หัวข้อ 2.1 พื้นที่ค้นหาหนึ่งล้าน, หัวข้อ 5.3 การยุบหลักหน้าเป็นช่วงบน key, หัวข้อ 5.4 การจองด้วยการสุ่มเลข

เอกสารออกแบบสำหรับโจทย์ Lottery Search System ฉบับนี้เป็นภาษาไทย ส่วนฉบับภาษาอังกฤษ
อยู่ที่ DESIGN_DOCUMENT.md และ docs/DESIGN_DOCUMENT.html บทวิเคราะห์ประกอบอยู่ใน
docs/RESULTS.md และเอกสารวางแผนขยาย ส่วนข้อมูลดิบทั้งหมดอยู่ใน docs/evidence/